CADT IDT

# W10 - PRACTICE

## Monster Slayer

### At the end of this practice, you should be able to…

- ✓ Use all concepts you have learned so far to create a complex project

### How to work?

- ✓ Download **the start code** from the Google classroom
- ✓ For each exercise you can either:
  - Run `npm install`
  - Or move an existing `node_modules` to the exercise folder *(fastest option!)*

### How to submit?

- ✓ **Create a repository on GitHub** with the name of this practice:
  Ex: C2-S1-PRACTICE
- ✓ **Push your final code** on this GitHub repository (if you are lost, follow this tutorial)
- ✓ Finally, submit on **Google classroom** your GitHub repository URL
  Ex: `https://github.com/thebest/` C2-S1-PRACTICE`.git`

### Are you lost?

*You can read the following documentation to be ready for this practice:*
https://www.w3schools.com/react/react_jsx.asp
https://www.w3schools.com/react/react_props.asp
https://www.gatsbyjs.com/docs/how-to/images-and-media/importing-assets-into-files/

# *Understand the game rules*

Start with playing the game online, and understand its rules:
https://reactjs-game-practice.vercel.app/

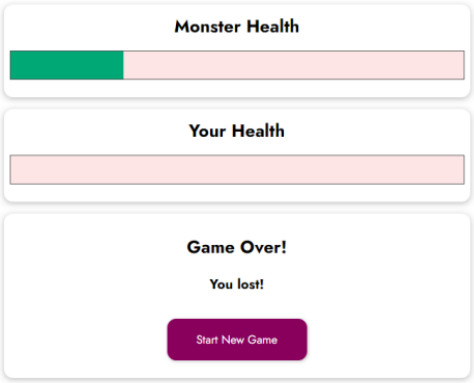
In this game you need to fight against a monster:

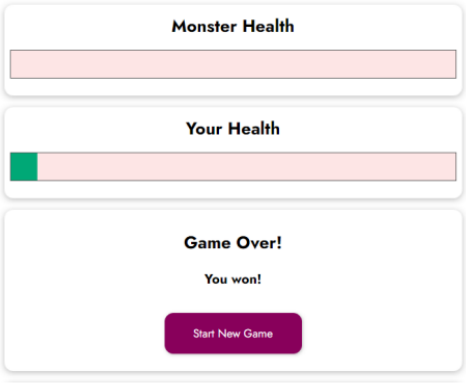
- You can choose to **attack** *(the monster will attack back)*
- Or **heal** yourself *(the monster will also attack back)*
- Every 3 turns, you can use a **special attack**
- You can also **kill** yourself if you want 😊


The game will also contain a dedicated component to display all actions:

Finally, the game over component will display the result (*you won, you lost, draw game*).

- The player can also **start a new game**




# *1. Vanilla HTML to React components*

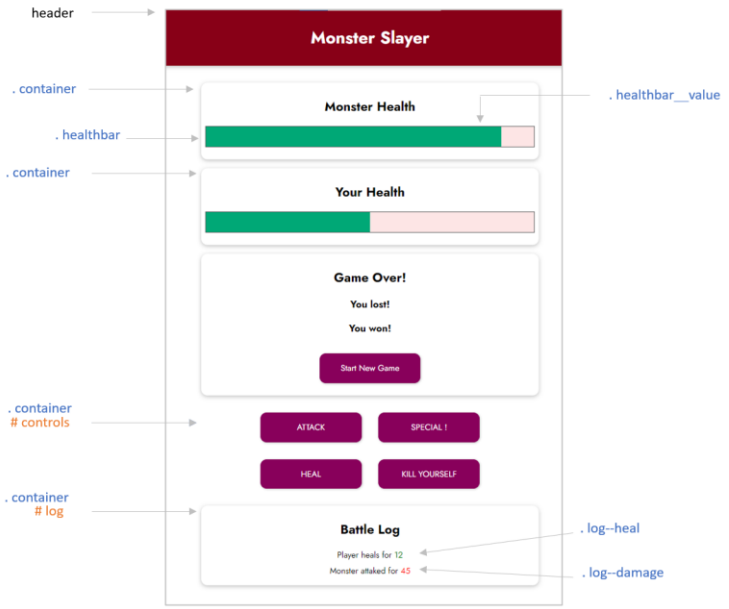
As start code we provide:

- A **vanilla HTML CSS** project, *which serves as example to style and structure your HTML*
- A **start REACT JS** project, *that will contain your code*

The goal of this first activity is only to **move** the HTML and CSS code from vanilla project to your new React JS components….

**Q1 –** First understand the vanilla HTML CSS project:

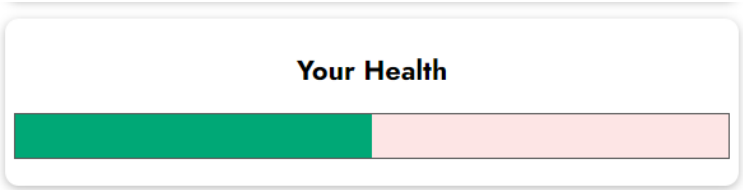
- *Which CLASSES are used?*
- *Which ID are used?*



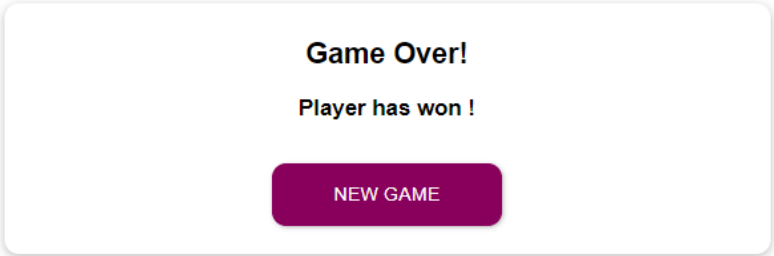

**Q2 –** Then, In your REACT JS project, create the components and move the HTML/CSS code

- **Header**: *representing the game header*

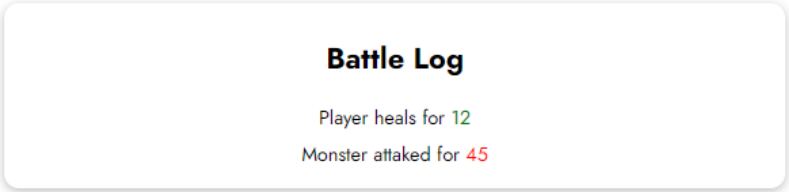

- **Game**: *containing all game logic and game sub components*
- **Entity**: *representing the monster or player's heath*



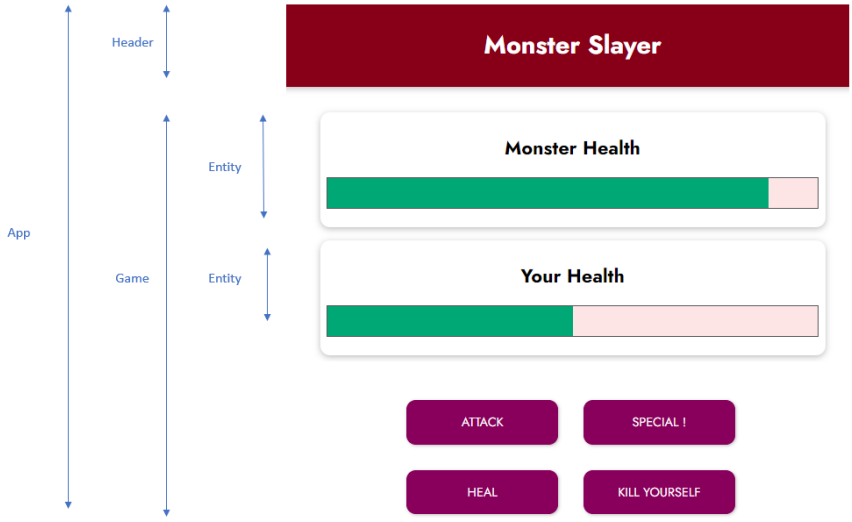

- **Game Over**: *informing about the end of the game:*




- **Log**: *representing all the game actions*




As a result, here is how your app components and its sub component could look like




**Q3 –** Draw the **component diagram** (power point) to represent how your components communicate.

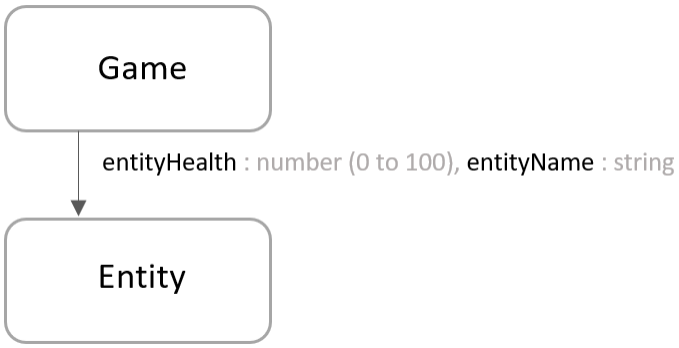
# *2. Entity component*

The **Entity** component gets 2 parameters:

- The health percentage *a number from 0 to 100*
- The entity name: either Player or Monster

**Q1 –** Complete your code

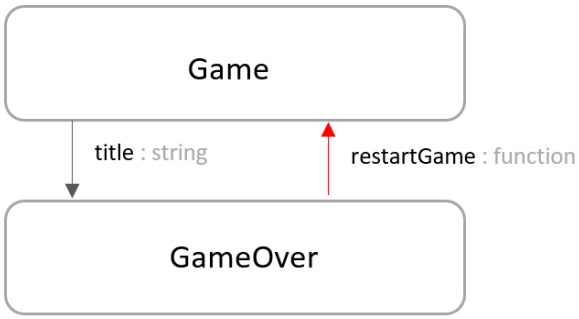
- To display the health bar properly in the Entity component
- To create 2 entities (monster and player) in the Game component




# 3. Game over component

The **GameOver** component gets 2 parameters:

- The title: *the game over message to display in the <h3>*
- The restartGame: *a function reference to be called when user click on start new game.*




# 4. Log component

Each log message can be stored using the following data structure:

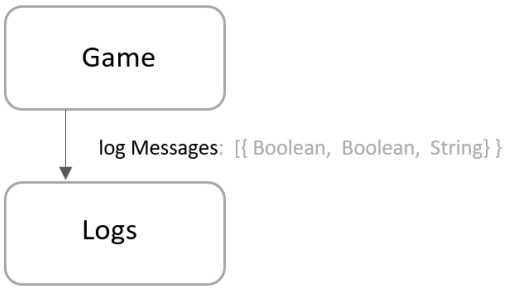
- **isPlayer**:  whether its s the player or the monster
- **isDammage**: whether it's a damage or a heal
- **text**: the text to display

| Log |
| --- |
| isPlayer: Boolean |
| isDamage: Boolean |
| text: String |

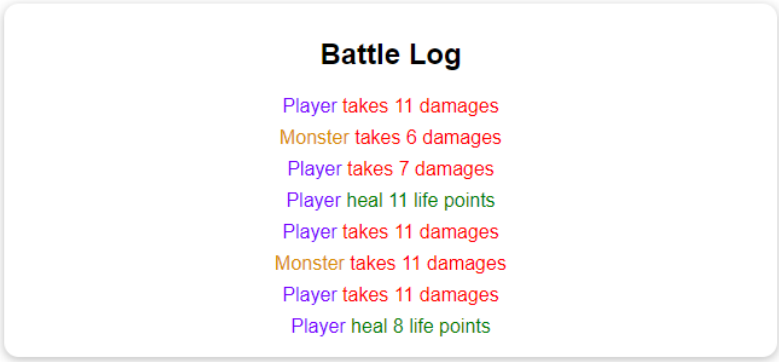
*Note: we already have created some helper function in the Game component to create log messages.*

The **Logs** component gets 1 parameter:

- The list of logs, following the above structure




The **Logs** component shall display each log, following the color specification:




# *5. Game component*

The **Game** component handles the game logic. It will be the **main component** of your project.

**Q1 –** Take time to list down the *states* and the *computed values* you will need to handle all your game features.

Note: Computed values are data that do not need a state, as they can be computed from other data (state, constant, variable.)

For example: how will you manage whether the game is over, who has won? Whether the special attack need to be displayed or not? How will you store the logs? Etc.

| State | Type | Role |
|---|---|---|
| logs | array | Action history |
| gameOver | boolean | Is game finished |
| playerHealth | number | Play HP |

| Computed values | Type | Role |
|---|---|---|
| winner | string | player |

| | | |
|---|---|---|
| canusespecial | boolean | Every 3 turns |
| Isgameover | boolean | Health <=0 |

**Q2 –** Link your states ad computed value to your component's attributes

- Visibility of the GameOver, the buttons
- The monster and player health bar
- The log component

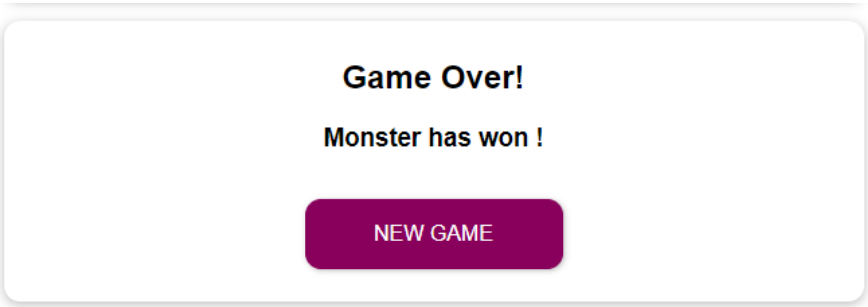
**Q3 –** Handle **actions** related to each button, following the **specifications**.

*Note: we already have created a helper function in the Game component to generate a random value between a min and a max value.*


**Q4 –** Handle **the end of the game**

The GameOver component shall display when the game is over, and propose the player to start a new game

*Note: What do you need to update to start a new game?*




# 6. Test and deploy on Vercel

**Q1 –** You game should handle **every situation**

For instance, did you think about those difference cases?

- What s happen in case of draw game?
- Can player or monster have more than 100% life?
- Can player or monster have less than 0% life?

**Q2 –** Before deploying, clean up your code

- Each function needs comments
- Remove useless code, useless variables…

**Q3 –** Push your code on GitHub and deploy on Vercel (search on internet how to deploy)